

Pros and Cons of ML Models

A quick-reference guide to common machine learning algorithms

This guide summarizes the practical strengths and weaknesses of widely used machine learning algorithms, helping you choose the right model for a given problem at a glance.

Algorithms covered:

- SVM (Support Vector Machine)
 - Naive Bayes
 - Logistic Regression
 - Random Forest
 - Decision Trees
 - KNN (K-Nearest Neighbours)
 - XGBoost
 - K-Means
-

SVM (Support Vector Machine)

Pros

- Performs well in higher dimensions
- Best algorithm when classes are separable
- Outliers have less impact
- Well suited for extreme-case binary classification

Cons

- Slow: requires a large amount of time to process larger datasets
- Poor performance with overlapping classes
- Selecting appropriate hyperparameters is important
- Selecting the appropriate kernel function can be tricky

Naive Bayes

Pros

- Real-time prediction
- Scalable with large datasets
- Insensitive to irrelevant features
- Supports multi-class prediction
- Good performance

Cons

- The assumption of feature independence rarely holds
- Bad estimator: probability outputs from `predict_proba` should not be taken too seriously
- Training data should represent the population well

Logistic Regression

Pros

- Simple to implement
- Effective
- Feature scaling not needed
- Tuning of hyperparameters not needed

Cons

- Poor performance on non-linear data (e.g. image data)
- Poor performance with irrelevant and highly correlated features
- Not a very powerful algorithm

Random Forest

Pros

- Reduced risk of overfitting
- Good performance on imbalanced datasets
- Can handle huge amounts of data
- Good handling of missing data
- Little impact from outliers
- Useful for extracting feature importance

Cons

- Time-consuming process
- Requires more resources
- Appears as a black box

Decision Trees

Pros

- Normalization or scaling of data not needed
- Handles missing values
- Easy to explain
- Easy to visualize
- Automatic feature selection

Cons

- Prone to overfitting
- Sensitive to changes in data
- Higher time required to train

KNN (K-Nearest Neighbours)

Pros

- Simple to understand and implement
- No assumptions about the data (unlike linear regression, which assumes a linear relationship, or Naive Bayes, which assumes feature independence)
- Constantly evolving model
- Can solve multi-class problems
- Only one hyperparameter to tune
- No training involved ("lazy"); new training examples can be added easily

Cons

- Slow for large datasets
- Does not work very well on datasets with a large number of features
- Scaling of data is required
- Does not work well on imbalanced data
- Sensitive to outliers
- Cannot deal well with missing values

Complexity note

Expensive & slow: $O(md)$, where m = number of examples and d = number of dimensions. To find the nearest neighbour of a new point x , the distance to all m training examples must be computed. Runtime is slow but can be improved by: pre-sorting training examples into fast data structures, computing approximate distances, or removing redundant data (condensing).

XGBoost

Pros

- Less feature engineering required (no scaling/normalizing needed; handles missing values well)
- Feature importance can be found
- Fast to interpret
- Outliers have minimal impact
- Handles large datasets
- Good execution speed
- Good model performance
- Less prone to overfitting

Cons

- Difficult to interpret; visualization is tough
- Harder to tune due to the large number of hyperparameters

K-Means

Pros

- Relatively simple to implement
- Scales to large datasets
- Guarantees convergence
- Can warm-start the position of centroids
- Easily adapts to new examples
- Generalizes to clusters of different shapes and sizes (e.g. elliptical clusters)

Cons

- Choosing k manually
- Dependent on initial values
- Struggles clustering data of varying sizes and density
- Sensitive to clustering outliers
- Scaling with the number of dimensions becomes difficult